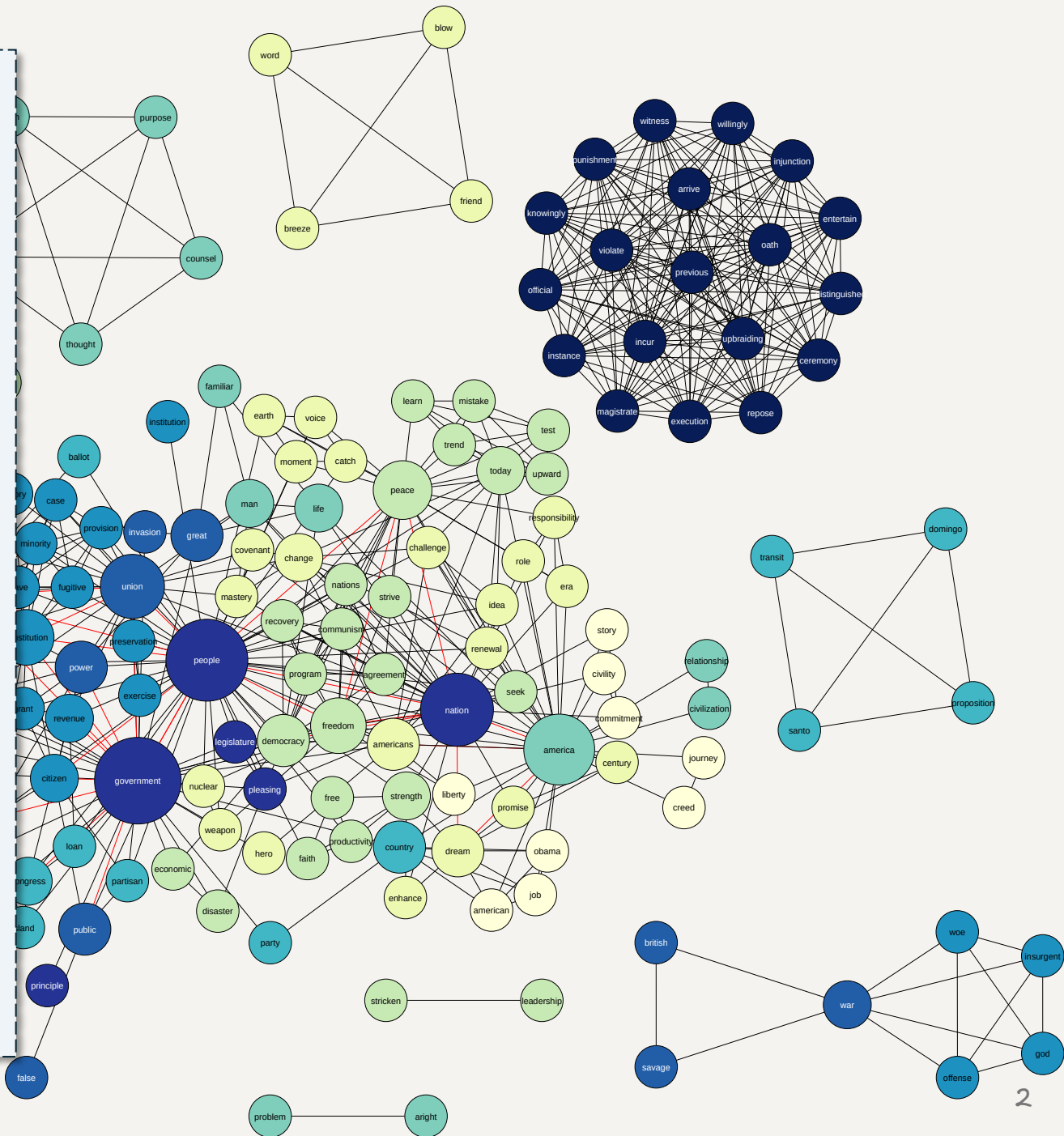


Graph-Based Analysis of the US Presidents' Inauguration Addresses 1789 – 2017

Agenda

- 01 Key concept:
Co-occurrence
- 02 From corpus to keywords
- 03 From keywords to graph
- 04 From graph to visualization
- 05 Dashboard
 - Layout
 - Interactivity
 - Technology stack
 - Some details



Key Concept: Co-occurrence

Informal definition:

Entities sharing some context are said to co-occur.

In a co-occurrence graph, where entities are represented as nodes, edges between these nodes show their co-occurrence.

Contexts may include:

- time
- space
- occasion
- publication (e.g. authorship)
- culinary preparation (e.g. ingredient list)
- board, community, club, association, ...
- materials (e.g. chemical elements)
- ...

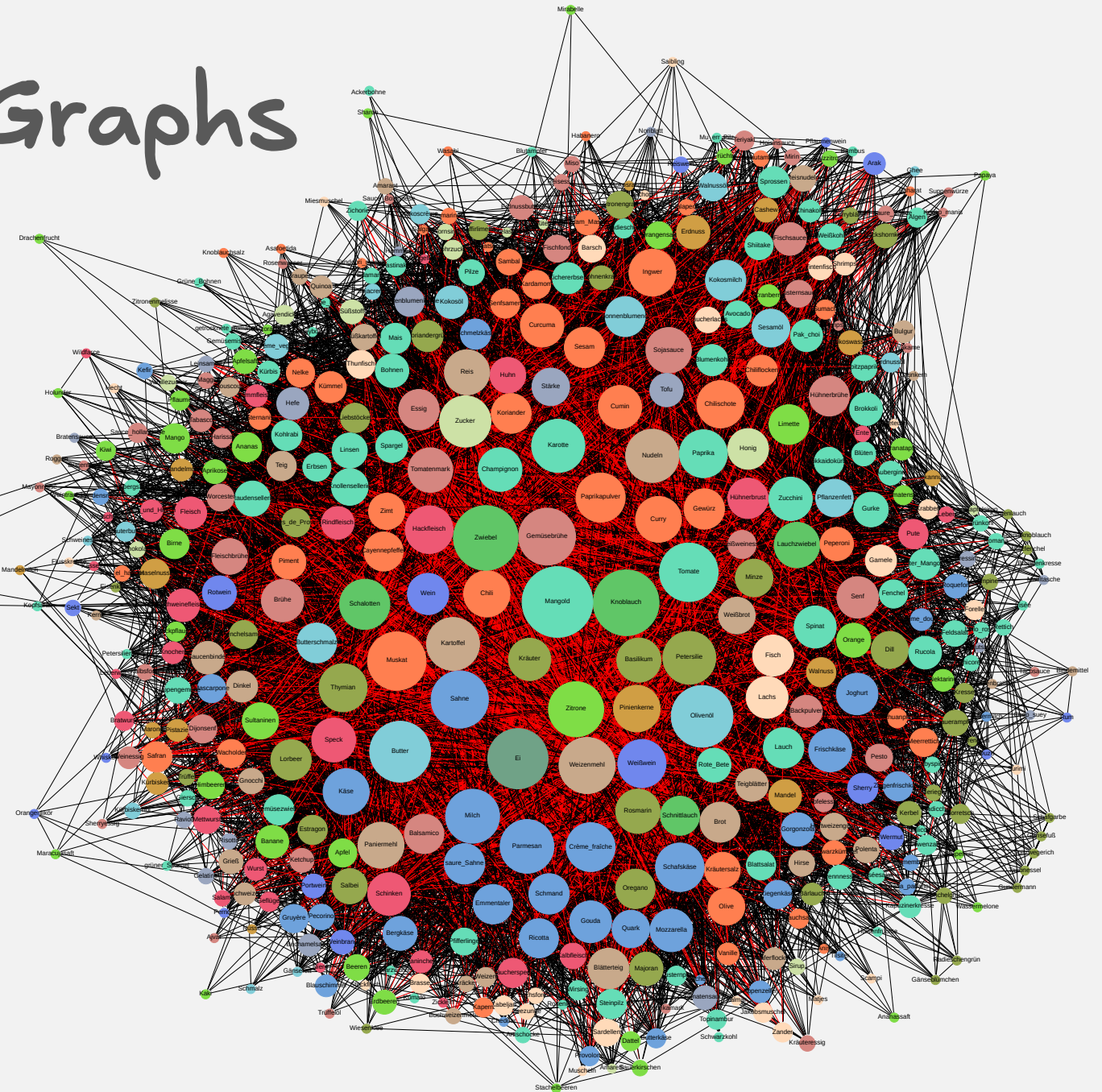
Co-occurrence Graphs

Ingredient co-occurrence graph for
"Mangold" recipes from chefkoch.de

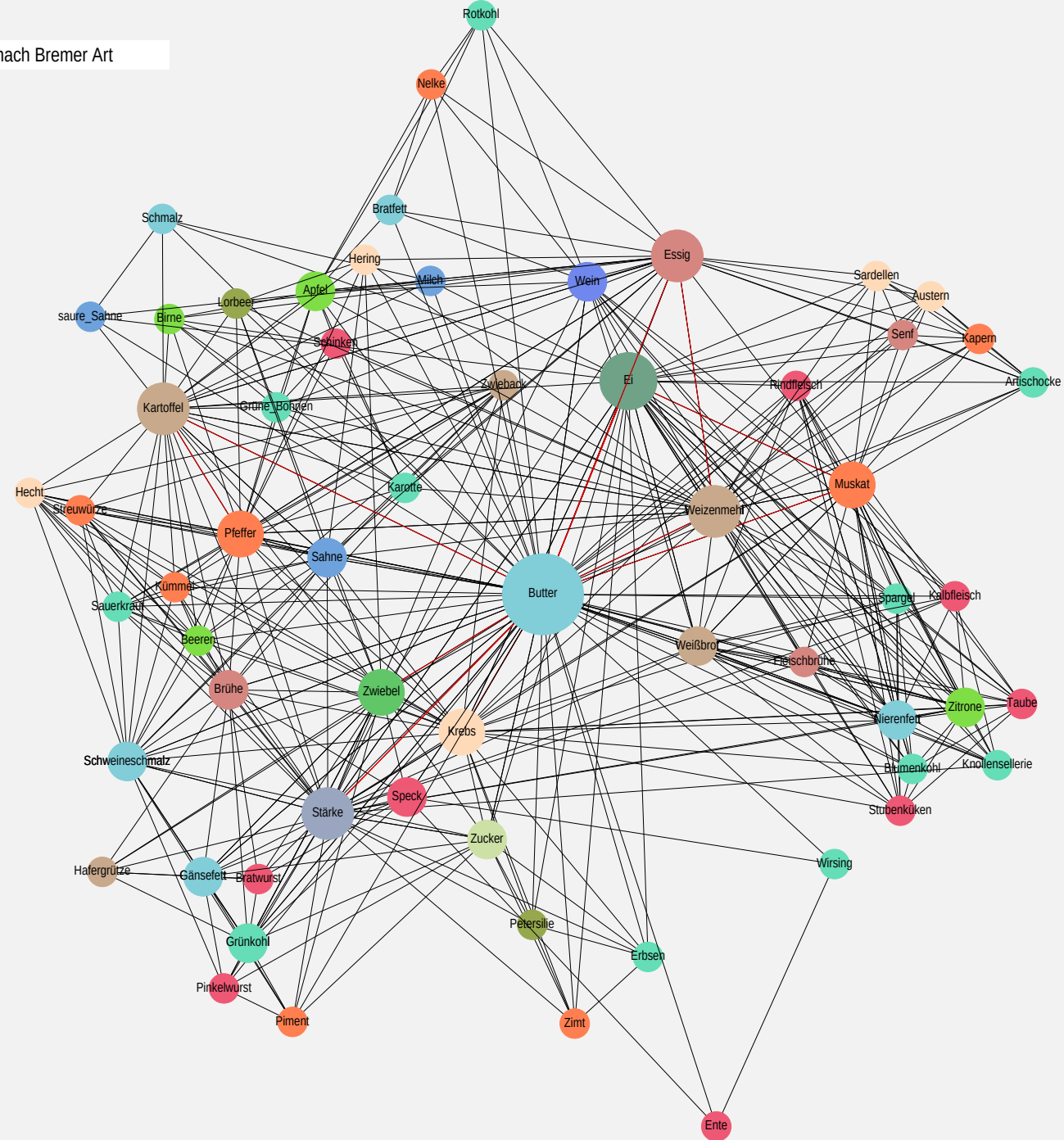
ingredients $\rightarrow$ nodes

Same-recipe usage $\rightarrow$ edges

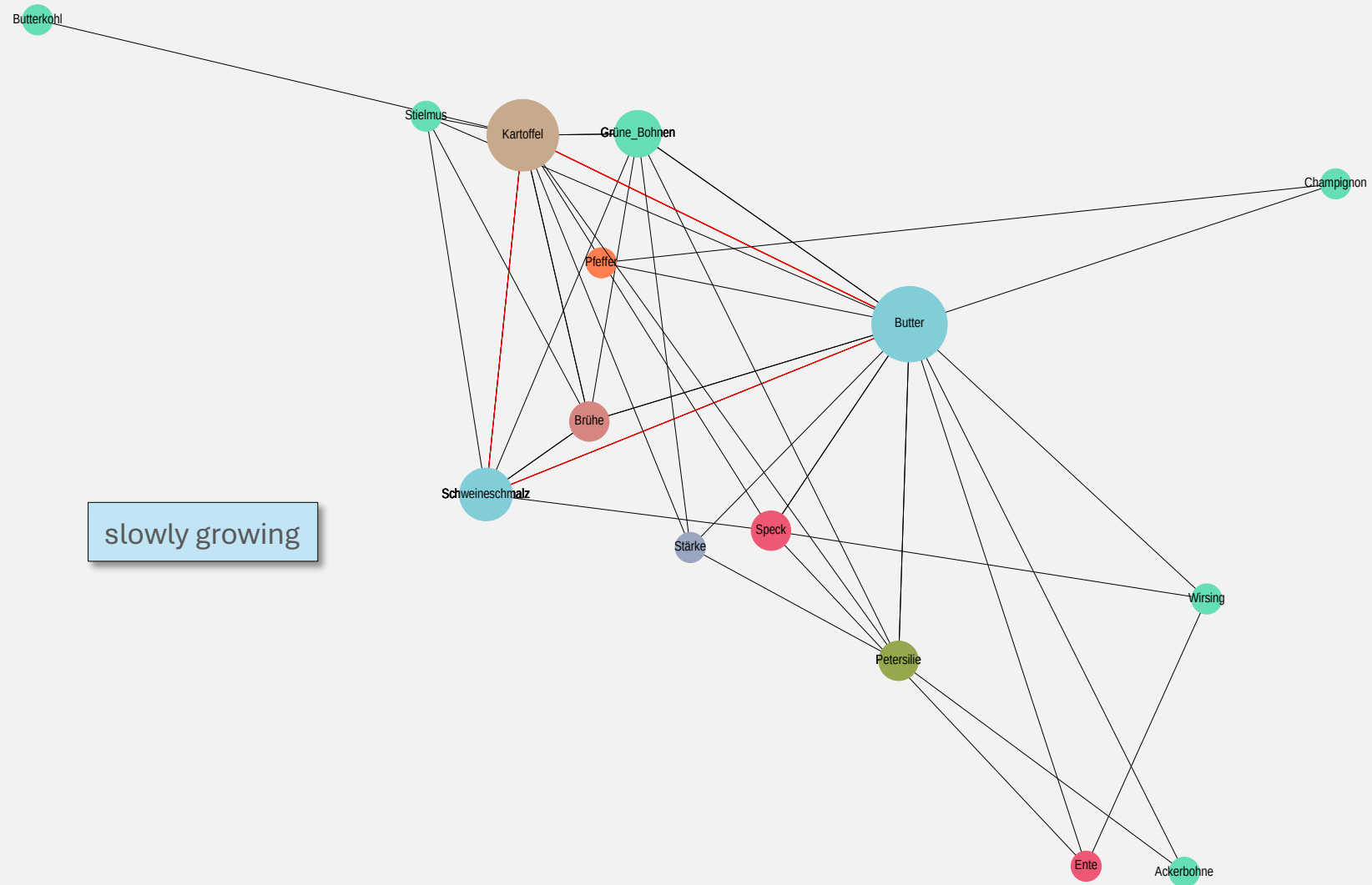
additional coloring and sizing



12 Grüner oder brauner Kohl ganz gekocht nach Bremer Art



fast growing



Co-occurrence Graphs (2)

Character co-occurrence graph for "Les misérables"

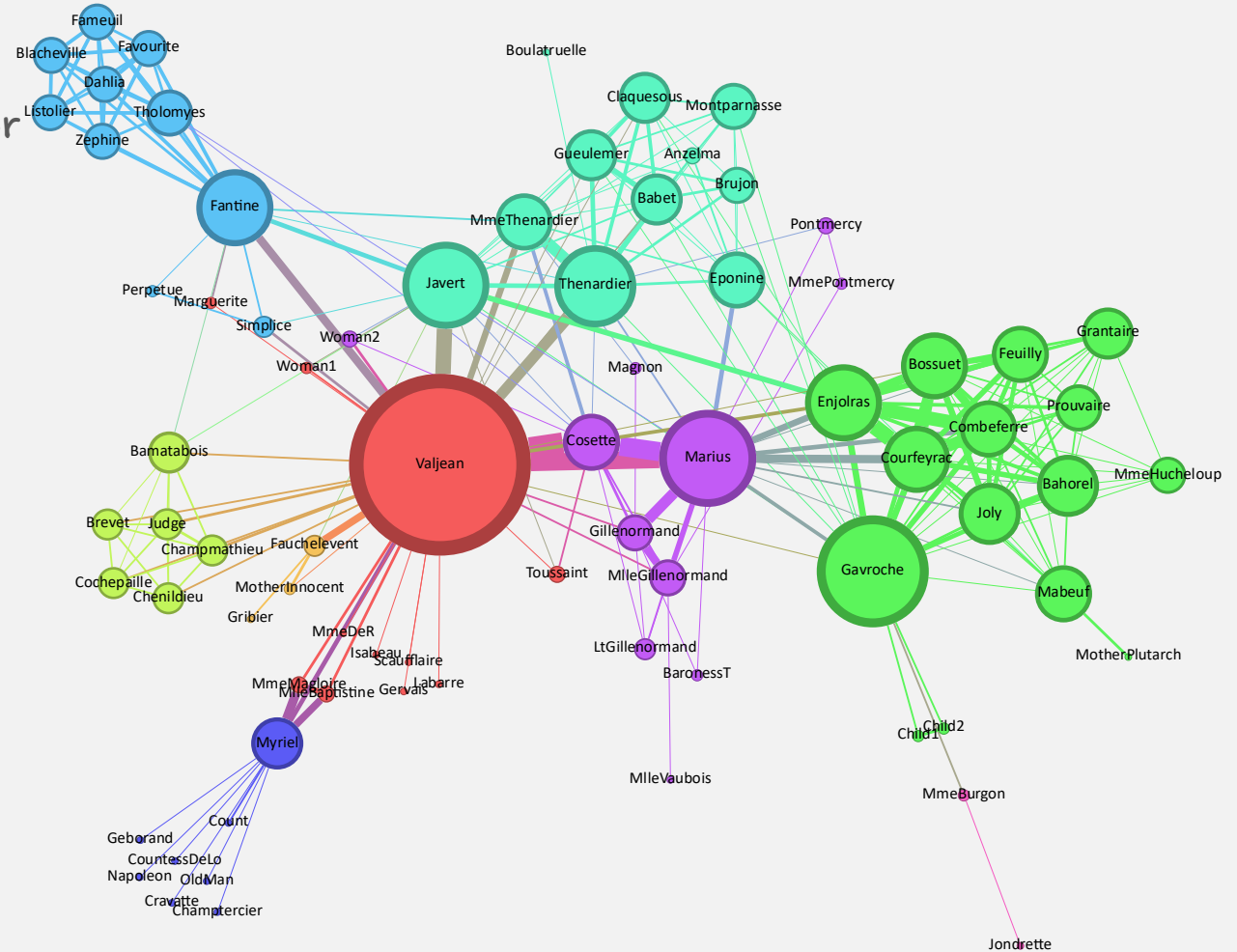
Theater characters/roles

→ nodes

Same-scene appearance

→ edges

here: additional
coloring and sizing



Co-occurrence Graphs (3)

Further Examples:

- companies sharing board members (Seierstad and Opsahl 2011),
- users meeting at locations or events (Eagle and Pentland 2006),
- users rating different products (Ziegler et al. 2005),
- consumers making payments to merchants (Martens and Provost 2011),
- mobile devices visiting locations (Provost et al. 2012, 2015),
- authors collaborating on scientific papers (Newman 2001b),
- people communicating on online forums (Opsahl 2011),
- actors playing in the same movies (Guillaume and Latapy 2006),
- words occurring in the same sentence/search query (Guillaume and Latapy 2006; Cancho and Solé 2001),
- proteins involved in the same metabolic processes (Guillaume and Latapy 2006),
- **Wer redet über was im Bundestag?** (Banisch, DGNNet-Kongress 2022, Darmstadt),
- ...

Stankova, Marija; Praet, Stiene; Martens, David; Provost, Foster (2021):
Node classification over bipartite graphs through projection.
Machine Learning, 110. DOI: 10.1007/s10994-020-05898-0.

Co-occurrence Graphs (4)

A more formal definition:

"Given a concept set $\mathcal{C} = \{c_i\}_{i=1}^N$ consisting of N concepts

and a paper corpus $P = \{p_i\}_{i=1}^M$ consisting of M papers,

let $\mathcal{C}_p \subset \mathcal{C}$ denote the set of concepts paper $p \in P$ contains.

When concepts c_u and c_v ($c_u \neq c_v$) occur together in the same paper $p \dots$,

i.e., $c_u \in \mathcal{C}_p \wedge c_v \in \mathcal{C}_p$, it is considered that c_u and c_v co-occur,

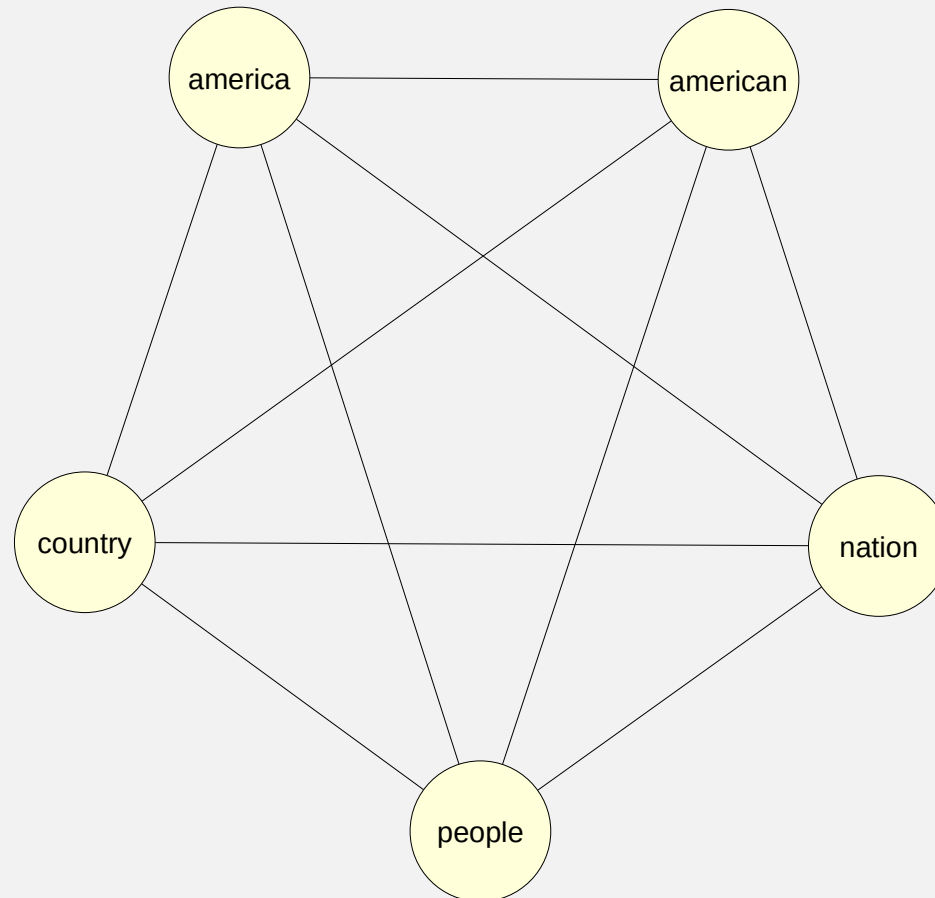
that is, there is a connection between the two concepts."

Gu, Y., Dong, L., Wei, F., & Huang, M. (2023).
Pre-Training to Learn in Context.
In A. Rogers, J. Boyd-Graber, & N. Okazaki (Hrsg.),
*Proc. 61st Annual Meeting of the Association for
Computational Linguistics*(S. 4849–4870).

Co-occurrence Graphs (5)

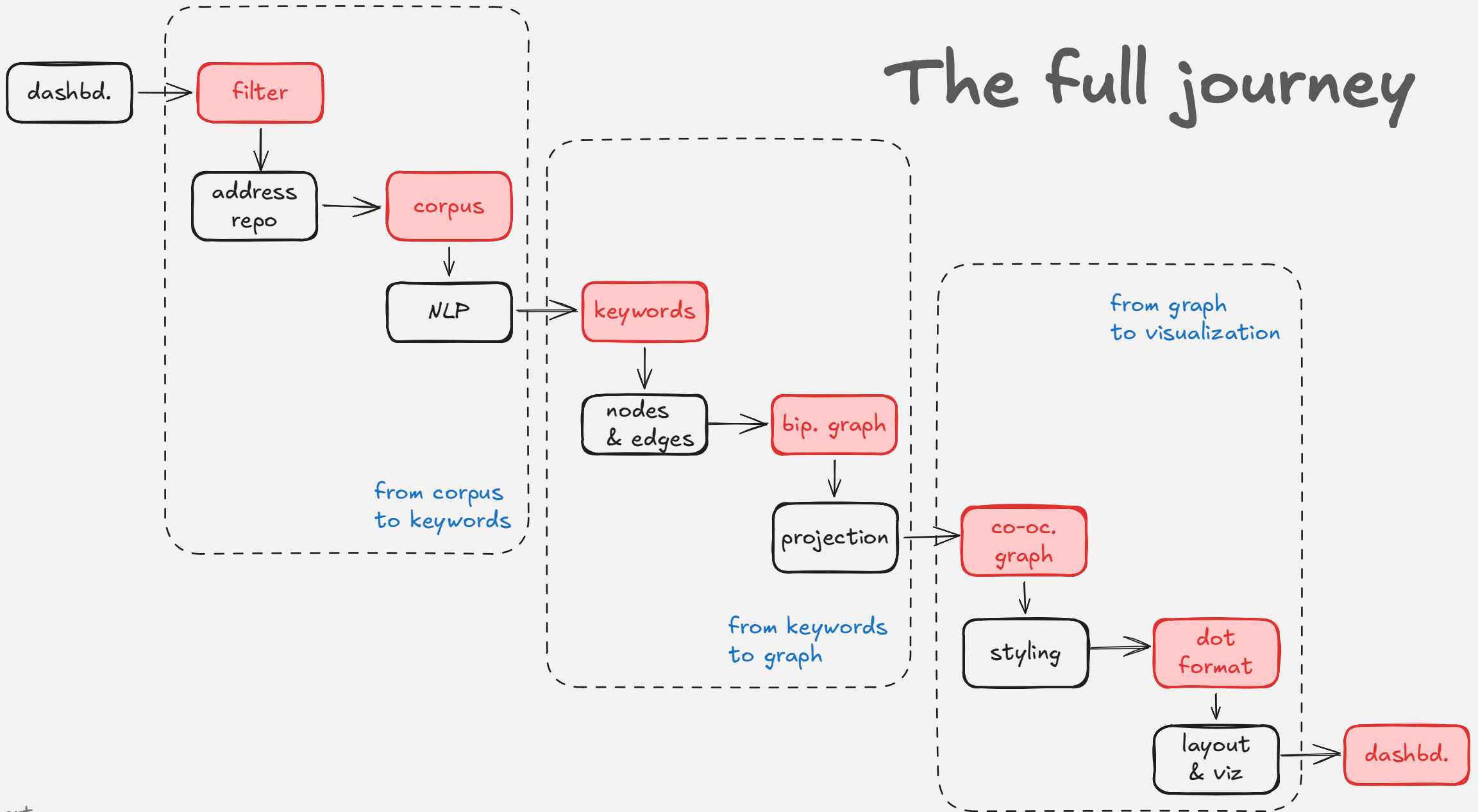
Application view:
"co-occurrence"

Graph theory view:
"complete graph"

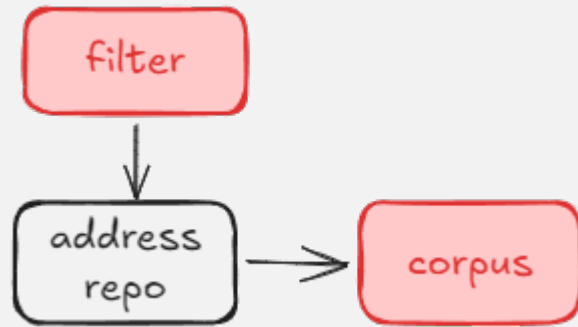


A **complete graph** (or "clique") is a graph in which each pair of vertices is connected by an edge, i.e., a graph in which each two vertices are adjacent. The complete graph with n vertices is denoted K_n . It has $\binom{n}{2} = \frac{1}{2} n \cdot (n - 1)$ edges.

The full journey



Start



What is a Corpus?

"A **corpus** is a collection of texts. ...

... there is today a growing consensus that a corpus is a collection of machine-readable authentic texts sampled to be representative.

- Firstly, to say that they are **machine-readable** in effect means that the texts can be manipulated and searched with the help of a computer, using some kind of specialised interface.
- Secondly, to say that the texts are **authentic** means that they have been taken from original sources of written and spoken language, such as published books, periodicals, reports, lectures, talks, meetings, speeches, sermons, and sport commentaries.
- Finally, to say that they are **representative** means that the collected texts should ideally represent a particular language variety. The language texts of a corpus are thus normally assembled with particular purposes in mind."

<https://www.awelu.lu.se/language/corpora-resources-for-writer-autonomy/what-is-a-corpus/>

The Address Repository (2)

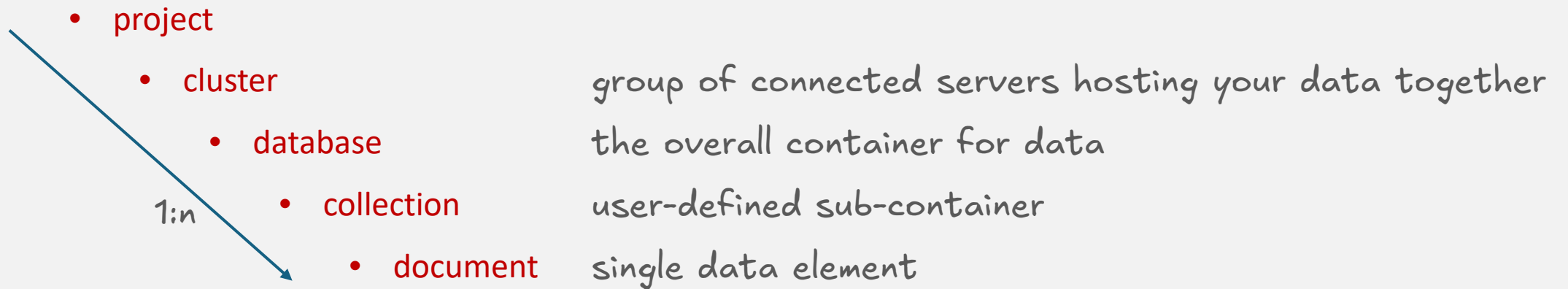
Storage for the full set of inauguration addresses



- document-oriented database
- JSON-based interaction for querying, inserting, or updating records
- comes with a managed cloud service called MongoDB Atlas
- operates on AWS, Google Cloud Platform, and Microsoft Azure

The Address Repository (3)

MongoDB terminology hierarchy



You can separate your data into multiple databases per cluster or put everything into one database with multiple collections.

The Address Repository (3)

three address docs
in the Address Repo

```
_id: ObjectId('6a07069ba12e3e7dc893bf4d')  
addr_id: "1789_George_Washington"  
in_repo: "01"  
pres_name: "George Washington"  
party: "Independent"  
year: "1789"  
date: "1789-04-30"  
txt: "Fellow-Citizens of the Senate and of the House of Representatives: Am..."
```

```
_id: ObjectId('6a07069ba12e3e7dc893bf4e')  
addr_id: "1793_George_Washington"  
in_repo: "02"  
pres_name: "George Washington"  
party: "Independent"  
year: "1793"  
date: "1793-03-04"  
txt: "Fellow Citizens I AM again called upon by the voice of my country to e..."
```

```
_id: ObjectId('6a07069ba12e3e7dc893bf4f')  
addr_id: "1797_John_Adams"  
in_repo: "03"  
pres_name: "John Adams"  
party: "Federalist"  
year: "1797"  
date: "1797-03-04"  
txt: "WHEN it was first perceived, in early times, that no middle course for..."
```

document rendering by
MongoDB Compass

The Address Repository (4)

For corpus building,
we can select
address docs by
their tag values.

:: inaug_addresses	
🔍 _id	objectId
addr_id	string
date	string
in_repo	string
party	string
pres_name	string
txt	string
year	string

Building the Corpus

Example:

Get the address docs for the inauguration addresses held by presidents of the Democratic Party during the Cold War.

Query filter:

```
{ 'year' : { '$gte' : '1947', '$lte' : '1991' },  
  'party': { '$in' : ['Demokrat', 'Democrat'] }  
}
```

Building the Corpus (2)

```
db          = client.myDatabase
collection  = db.inaug_addresses
corpus      = list(collection.find(query_filter, {
    "_id": 0,
    "addr_id": 1,
    "pres_name": 1,
    "year": 1,
    "party": 1,
    "date": 1,
    "txt": 1
})))
```

Building the Corpus (3)

Resulting corpus structure is list of dicts:

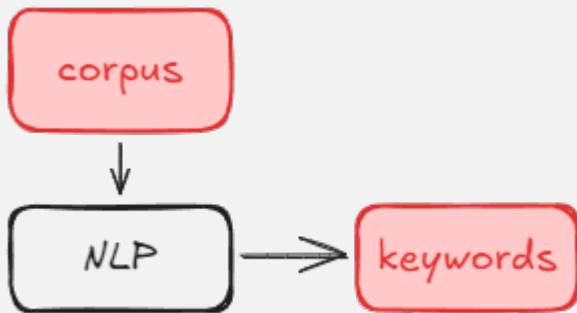
```
[
  {'addr_id': '1949_Harry_S._Truman', 'pres_name': 'Harry S. Truman', 'party': 'Democrat',
   'year': '1949', 'date': '1/20/1949', 'txt': "[Delivered in person at the Capitol] Mr. Vice President, ... "},
  {'addr_id': '1953_Dwight_D._Eisenhower', 'pres_name': 'Dwight D. Eisenhower', 'party': 'Republican',
   'year': '1953', 'date': '1/20/1953', 'txt': "My friends, before I begin ... "},
  {'addr_id': '1957_Dwight_D._Eisenhower', 'pres_name': 'Dwight D. Eisenhower', 'party': 'Republican',
   'year': '1957', 'date': '1/21/1957', 'txt': "Mr. Chairman, Mr. Vice President, Mr. Chief Justice, ... "},
  ...
]
```

Building the Corpus ⁽⁴⁾

Processing the corpus:

```
for address in corpus:  
    address_text = address['txt']  
    ...
```

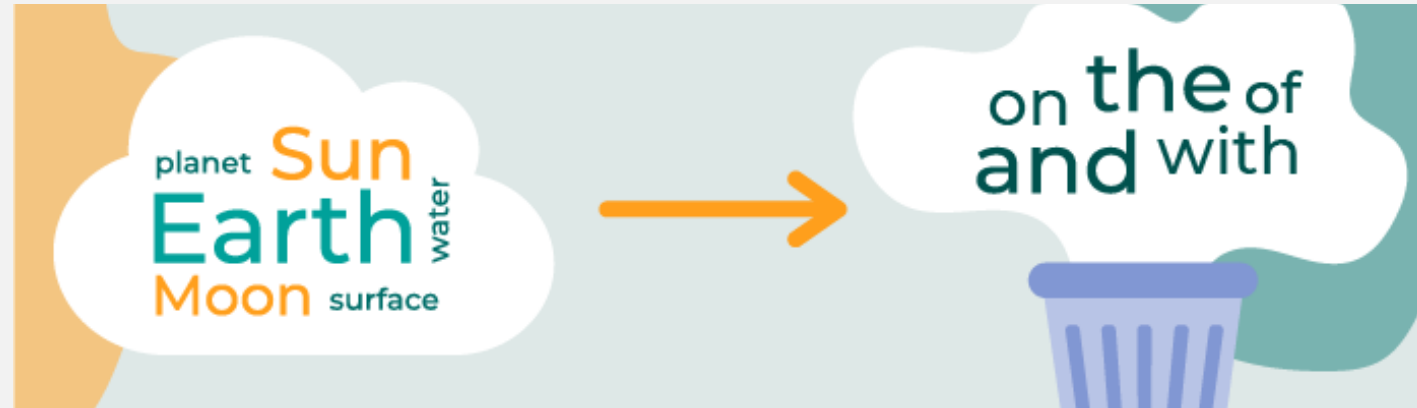
Natural Language Processing



NLP in 3 steps:

- stopwords removal
- lemmatization
- text vectorization (tf-idf)

Stopword elimination



<https://openclassrooms.com/en/courses/6532301-introduction-to-natural-language-processing/8080062-remove-stopwords-from-a-block-of-text>

Wikipedia: "Stopwords are high-frequency, low-semantic value words—such as "the," "is," "and," or "in"—filtered out during natural language processing (NLP) and text analysis. Removing them reduces noise, improves model efficiency, and helps focus on meaningful, informative words. Common stopwords lists vary by language and context."

Lemmatization

"In computational linguistics, lemmatization is the algorithmic process of determining the lemma of a word based on its intended meaning. Unlike stemming, lemmatization depends on correctly identifying the intended part of speech and meaning of a word in a sentence, as well as within the larger context surrounding that sentence, such as neighbouring sentences or even an entire document. As a result, developing efficient lemmatization algorithms is an open area of research."

<https://en.wikipedia.org/wiki/Lemmatization>



Lemmatization (2)

likes



like

better



good

worse



bad

<https://databasecamp.de/daten/stemming-lemmatizations>

Vectorization

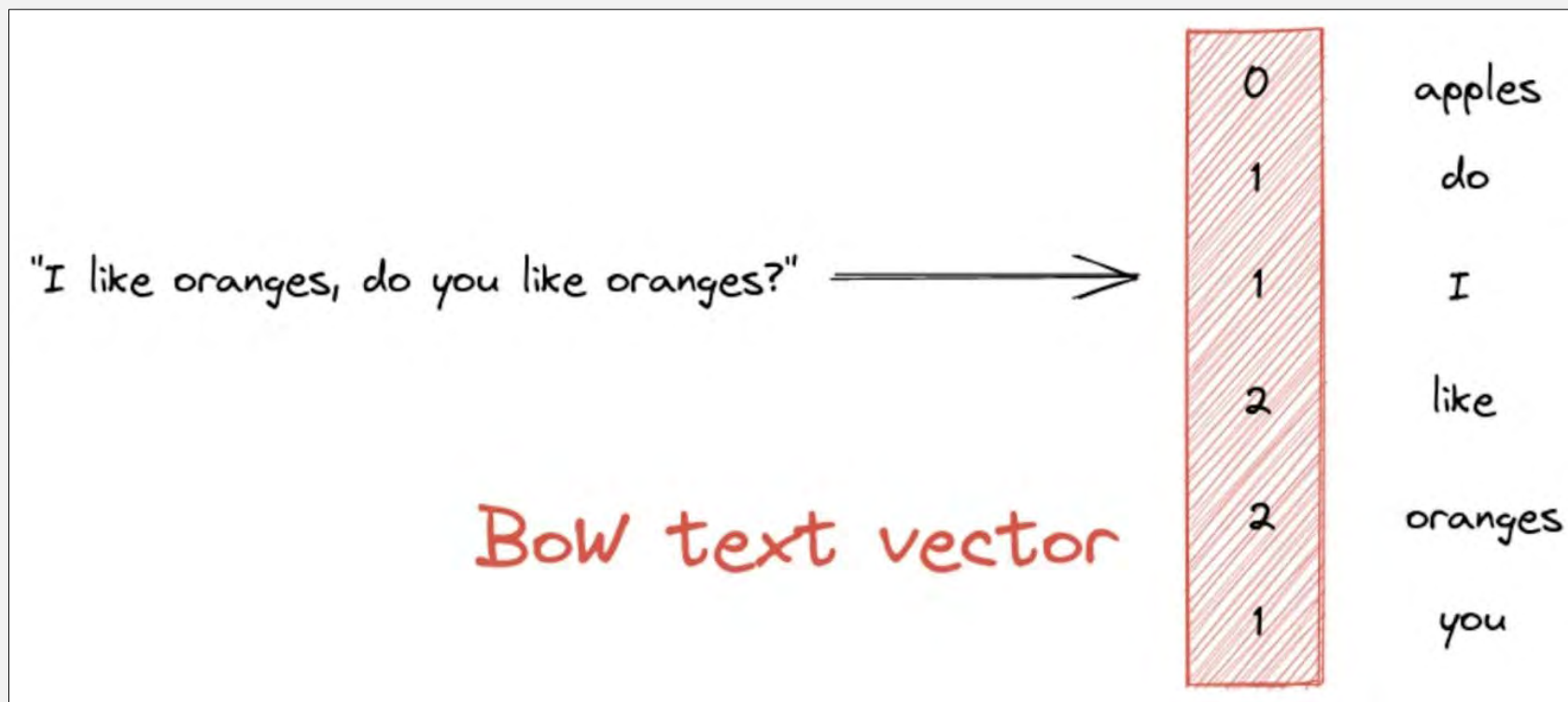
Text vectorization is a process used in natural language processing (NLP). It represents words or phrases as numerical values that capture their meanings and relationships within the text.

Common methods of text vectorization include:

- Bag of Words (BoW):
Represents text by the frequency of words.
- Term Frequency—Inverse Document Frequency (TF-IDF):
Assigns a score to words based on their importance in a text (document) relative to a corpus.
- Word Embeddings (e.g., Word2Vec, GloVe):
Captures semantic relationships between words by placing them in a continuous vector space.



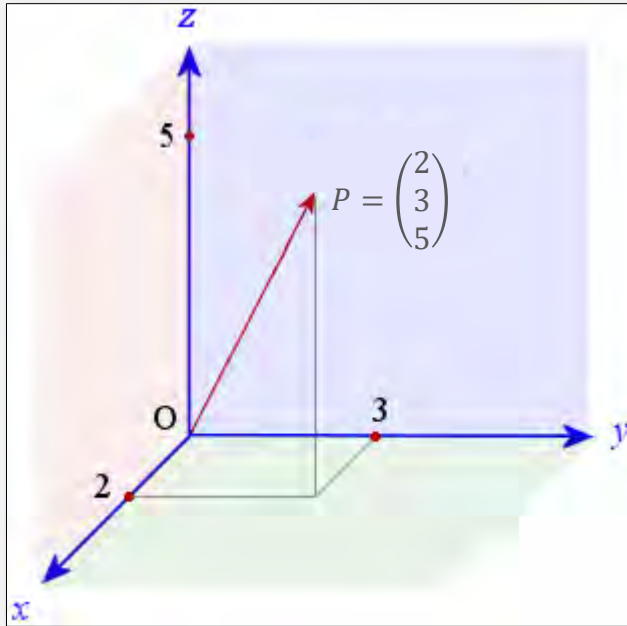
Vectorization: Bag-of-Words



<https://www.deepset.ai/blog/what-is-text-vectorization-in-nlp>

Bow: a simple approach based on counting words

Interlude: Vectors in space



<https://www.intmath.com/vectors/7-vectors-in-3d-space.php>

Try to:

- enhance from 3D to nD
- re-label from x-y-z to apples-oranges-like- ...

Vectorization: TF-IDF

Term Frequency – Inverse Document Frequency

"TF-IDF is a numerical statistic that reflects the **significance** of a word within a document **relative to** a collection of documents, known as a **corpus**. The idea behind TF-IDF is to quantify the importance of a term in a document with respect to its **frequency** in the document and its **rarity** across multiple documents."

<https://medium.com/@abhishekjainindore24/tf-idf-in-nlp-term-frequency-inverse-document-frequency-e05b65932f1d>

Vectorization: TF-IDF (2)

Term Frequency — Inverse Document Frequency

$tf(t, d)$ is the relative frequency of term t within document d

$$tf(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

The inverse document frequency measures how common or rare a term is across all documents. It is the logarithmically scaled inverse fraction of the number of documents that contain the term.

$$idf(t, D) = \log \frac{|D|+1}{|\{d \mid d \in D \wedge t \in d\}|+1} + 1$$

(Definition following
Python TfidfTransformer default)



$$tfidf(t, d, D) = tf(t, d) \cdot idf(t, D)$$

Comparing Bow and TF-IDF

entchen = "Alle meine Entchen schwimmen auf dem See,
schwimmen auf dem See, Köpfchen in das Wasser,
Schwänzchen in die Höh."

täubchen = "Alle meine Täubchen gurren auf dem Dach,
gurren auf dem Dach, fliegt eins in die Lüfte,
fliegen alle nach."

texts = [entchen, täubchen]

Bow	Entchen	Täubchen
alle	1	2
auf	2	2
dach	0	2
das	1	0
dem	2	2
die	1	1
eins	0	1
entchen	1	0
fliegen	0	1
fliegt	0	1
gurren	0	2
höh	1	0
in	2	1
köpfchen	1	0
lüfte	0	1
meine	1	1
nach	0	1
schwimmen	2	0
schwänzchen	1	0
see	2	0
täubchen	0	1
wasser	1	0

TF-IDF	Entchen	Täubchen
alle	0.153115	0.306229
auf	0.306229	0.306229
dach	0.000000	0.430395
das	0.215197	0.000000
dem	0.306229	0.306229
die	0.153115	0.153115
eins	0.000000	0.215197
entchen	0.215197	0.000000
fliegen	0.000000	0.215197
fliegt	0.000000	0.215197
gurren	0.000000	0.430395
höh	0.215197	0.000000
in	0.306229	0.153115
köpfchen	0.215197	0.000000
lüfte	0.000000	0.215197
meine	0.153115	0.153115
nach	0.000000	0.215197
schwimmen	0.430395	0.000000
schwänzchen	0.215197	0.000000
see	0.430395	0.000000
täubchen	0.000000	0.215197
wasser	0.215197	0.000000

Scoring Terms

TF-IDF	Entchen	Täubchen
alle	0.153115	0.306229
auf	0.306229	0.306229
dach	0.000000	0.430395
das	0.215197	0.000000
dem	0.306229	0.306229
die	0.153115	0.153115
eins	0.000000	0.215197
entchen	0.215197	0.000000
fliegen	0.000000	0.215197
fliegt	0.000000	0.215197
gurren	0.000000	0.430395
höh	0.215197	0.000000
in	0.306229	0.153115
köpfchen	0.215197	0.000000
lüfte	0.000000	0.215197
meine	0.153115	0.153115
nach	0.000000	0.215197
schwimmen	0.430395	0.000000
schwänzchen	0.215197	0.000000
see	0.430395	0.000000
täubchen	0.000000	0.215197
wasser	0.215197	0.000000

Consider these tf-idf values as "term scores":

Scores beyond a threshold designate terms as keywords.

pandas df holding term scores

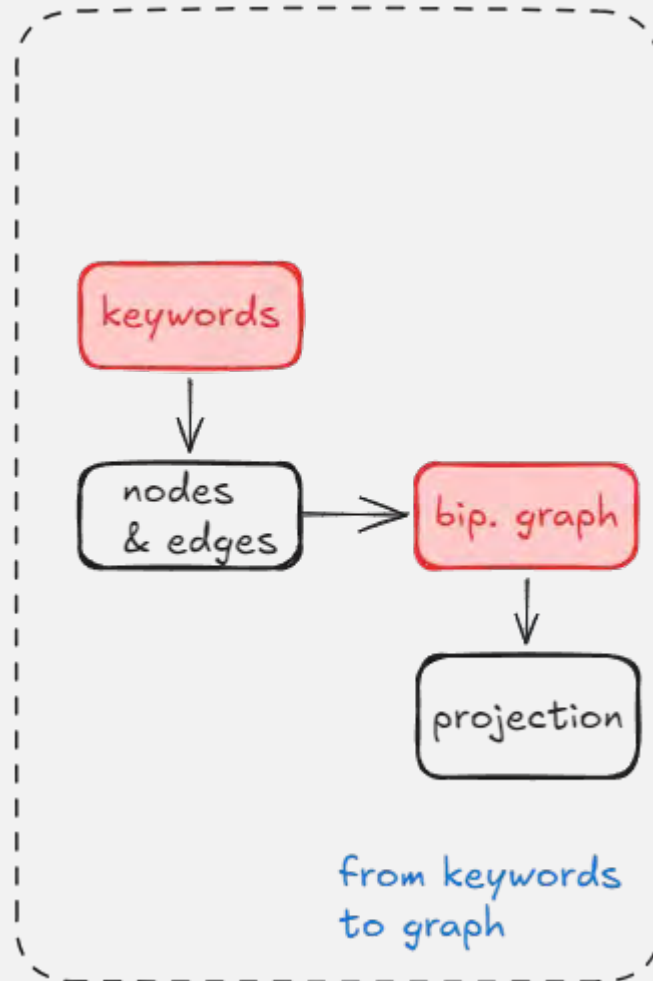
```
# create tf-idf vector per address
tfidf_vectorizer = TfidfVectorizer() # create vectorizer object
tfidf_vectors = tfidf_vectorizer.fit_transform(lemmatized_address_texts) # create vectors

# Convert the vectorizer into a pandas dataframe
# address IDs → row indices,
# terms ("feature names") → column indices
tfidf_df = pd.DataFrame(tfidf_vectors.toarray(), index=addr_ids, columns=tfidf_vectorizer.get_feature_names_out())
```

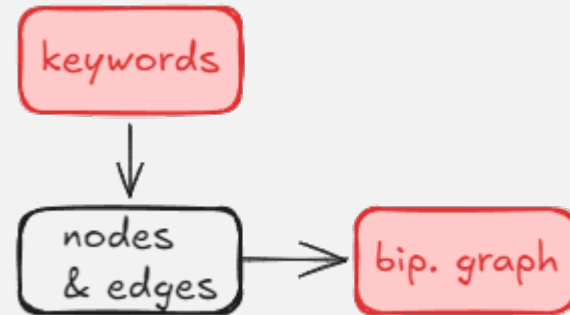
Resulting "address-to-keywords" dataframe for Roosevelt WW-2 inauguration addresses

	abandonment	achieve	achievement	action	age	agree	...	woman	word	work	write	year
1941_Franklin_D._Roosevelt	0.000000	0.022945	0.000000	0.032249	0.032249	0.000000	...	0.022945	0.032249	0.000000	0.064498	0.137673
1945_Franklin_D._Roosevelt	0.060796	0.086514	0.060796	0.000000	0.000000	0.060796	...	0.043257	0.000000	0.121592	0.000000	0.086514

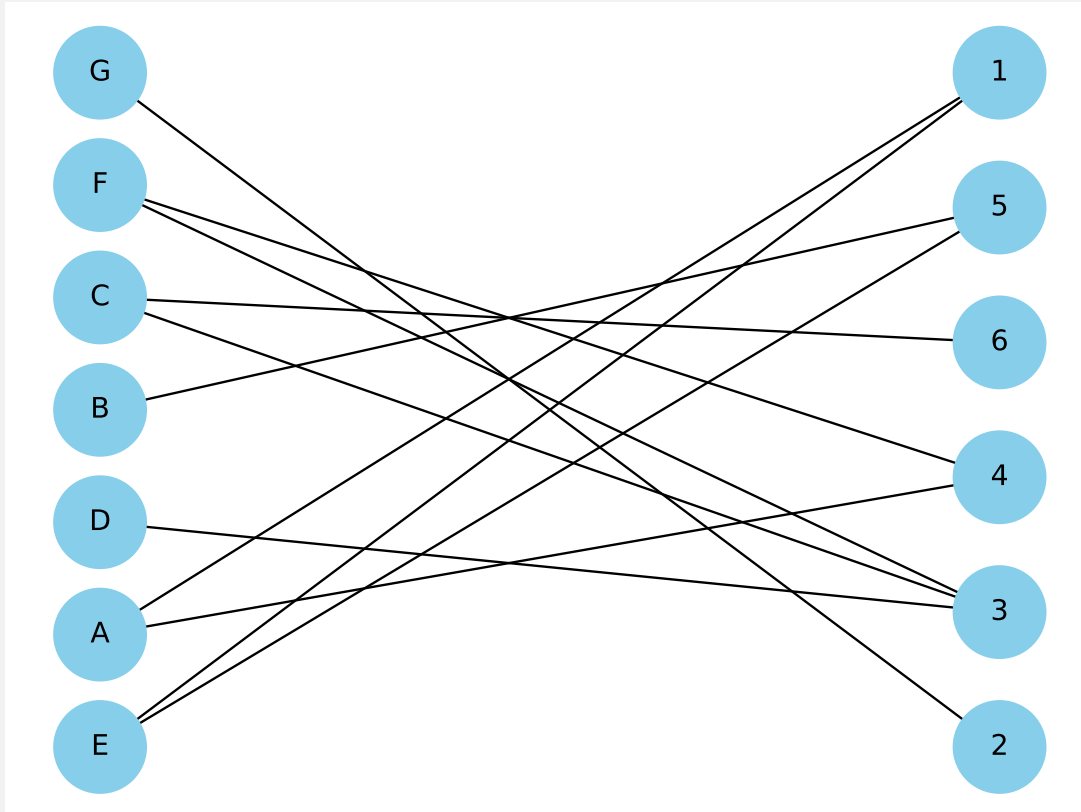
And now ...



An intermediate step



Bipartite Graphs



properties:

- o two independent sets of nodes, here marked as "letter nodes" (left) and "num nodes" (right)
- o no two letter nodes are connected
- o no two num nodes are connected
- o connections only between nodes from different sets

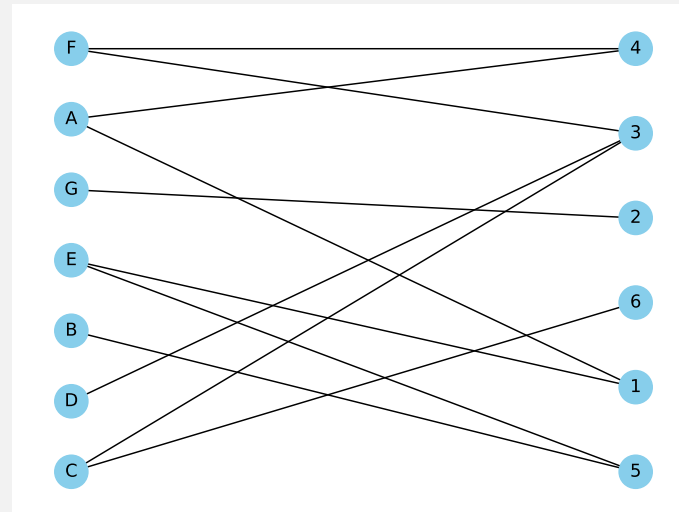
Why Bipartite Graphs?

We can easily construct a bipartite graph from the address-to-keywords dataframe.

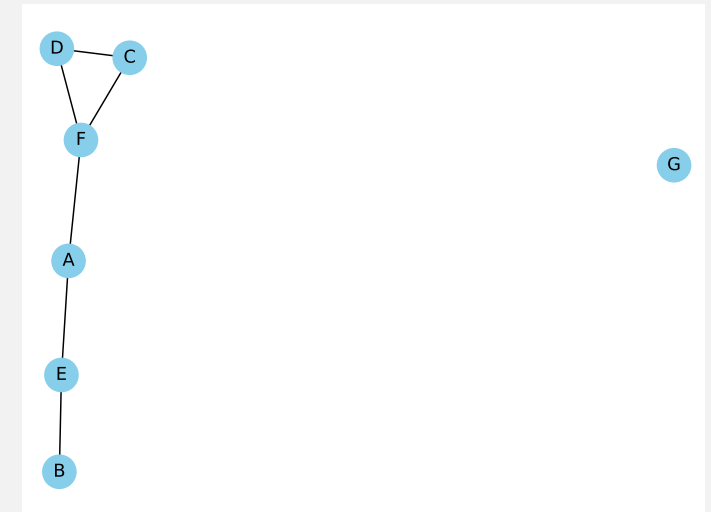


We can easily "project" a bipartite graph onto one of its node sets.

See next slides!



→
projection



Trick: Stacking & Thresholding

```
# Reshape the tfidf DataFrame so that the terms are in a single column.  
stacked_df = tfidf_df.stack().reset_index().rename(columns={0:'tfidf', 'level_0': 'address','level_1': 'term'})  
  
# Set threshold for TF-IDF values and determine remaining terms  
thres_tfidf = stacked_df[stacked_df['tfidf'] >= float(keyword_score)]
```

Dataframe stacking returns a reshaped DataFrame having a multi-level index with one or more new inner-most levels compared to the current DataFrame. The new inner-most levels are created by pivoting the columns of the current dataframe.

Stacking

row and
column index

	x	y	z
a	0	1	2
b	3	4	5
c	6	7	8



two-level
row index
i0, i1

i0	i1	val
a	x	0
a	y	1
a	z	2
b	x	3
b	y	4
b	z	5
c	x	6
c	y	7
c	z	8

Thresholding

i0	i1	val
a	x	0
a	y	1
a	z	2
b	x	3
b	y	4
b	z	5
c	x	6
c	y	7
c	z	8



threshold:
 $val > 3$

i0	i1	val
b	y	4
b	z	5
c	x	6
c	y	7
c	z	8

Stacked and thresholded dataframe

	address	term	tfidf
290562	1973_Richard_Nixon	role	0.234625
293692	1977_Jimmy_Carter	dream	0.185842
293856	1977_Jimmy_Carter	enhance	0.141087
295635	1977_Jimmy_Carter	nation	0.223362
297242	1977_Jimmy_Carter	strength	0.167142
298408	1981_Ronald_Reagan	americans	0.174071
300653	1981_Ronald_Reagan	government	0.174156
300819	1981_Ronald_Reagan	hero	0.182944
306747	1985_Ronald_Reagan	freedom	0.175981
306865	1985_Ronald_Reagan	government	0.183702
308131	1985_Ronald_Reagan	nuclear	0.172100
308395	1985_Ronald_Reagan	people	0.153520
310480	1985_Ronald_Reagan	weapon	0.145344
311199	1989_George_H._W._Bush	blow	0.172115
311246	1989_George_H._W._Bush	breeze	0.217588
312970	1989_George_H._W._Bush	friend	0.173483
316763	1989_George_H._W._Bush	word	0.155416
317041	1993_Bill_Clinton	america	0.353253
317044	1993_Bill_Clinton	americans	0.229144
317598	1993_Bill_Clinton	challenge	0.151375

node set 1

node set 2

NetworkX



- Data structures for graphs, digraphs, and multigraphs
- Many standard graph algorithms
- Network structure and analysis measures
- Generators for classic graphs, random graphs, and synthetic networks
- Nodes can be "anything" (e.g., text, images, XML records)
- Edges can hold arbitrary data (e.g., weights, time-series)
- Additional benefits from Python include fast prototyping, easy to teach, and multi-platform

...

Some NetworkX Details

A node is represented as tuple $(u, \{ 'att_x': '5' \})$, where u identifies the node and $'att_x'$ is an attribute.

An edge is represented as tuple $(u, v, \{ 'weight': 7 \})$, where u and v identify the nodes that the edge connects.

```
G = nx.Graph()
G.add_node('a', att_x=3)
G.add_edge('a', 'b', weight=5)
G.nodes(data=True) # show nodes including their attributes
G.edges(data=True) # same for edges
nx.degree(G)        # degree of nodes in graph G
G.degree()          # same
```

Constructing the Bipartite Graph

	address	term	tfidf
12	1941_Franklin_D._Roosevelt	america	0.183564
81	1941_Franklin_D._Roosevelt	democracy	0.206509
216	1941_Franklin_D._Roosevelt	life	0.229454
233	1941_Franklin_D._Roosevelt	measure	0.161245
251	1941_Franklin_D._Roosevelt	nation	0.344182
265	1941_Franklin_D._Roosevelt	people	0.183564
341	1941_Franklin_D._Roosevelt	speak	0.161245
342	1941_Franklin_D._Roosevelt	spirit	0.225743
631	1945_Franklin_D._Roosevelt	learn	0.303981
645	1945_Franklin_D._Roosevelt	man	0.173028
681	1945_Franklin_D._Roosevelt	peace	0.364777
787	1945_Franklin_D._Roosevelt	test	0.182389
792	1945_Franklin_D._Roosevelt	today	0.243185

node set 1

node set 2



```
# Create bipartite graph B from the thresholded DataFrame
B = nx.Graph()
for _, row in thres_tfidf.iterrows():
    address_node = row['address']
    term_node    = row['term']
    B.add_node(address_node, type='address')
    B.add_node(term_node,    type='term')
    B.add_edge(address_node, term_node)
```



Constructing the Bipartite Graph (2)

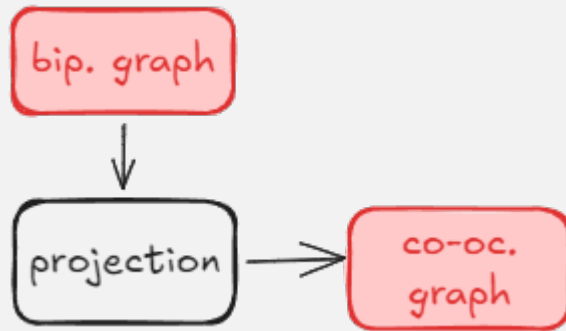
nodes

```
('1941_Franklin_D._Roosevelt', {'type': 'address'})
('america', {'type': 'term'})
('democracy', {'type': 'term'})
('life', {'type': 'term'})
('measure', {'type': 'term'})
('nation', {'type': 'term'})
('people', {'type': 'term'})
('speak', {'type': 'term'})
('spirit', {'type': 'term'})
('1945_Franklin_D._Roosevelt', {'type': 'address'})
('learn', {'type': 'term'})
('man', {'type': 'term'})
('peace', {'type': 'term'})
('test', {'type': 'term'})
('today', {'type': 'term'})
```

edges

```
('1941_Franklin_D._Roosevelt', 'america', {})
('1941_Franklin_D._Roosevelt', 'democracy', {})
('1941_Franklin_D._Roosevelt', 'life', {})
('1941_Franklin_D._Roosevelt', 'measure', {})
('1941_Franklin_D._Roosevelt', 'nation', {})
('1941_Franklin_D._Roosevelt', 'people', {})
('1941_Franklin_D._Roosevelt', 'speak', {})
('1941_Franklin_D._Roosevelt', 'spirit', {})
('1945_Franklin_D._Roosevelt', 'learn', {})
('1945_Franklin_D._Roosevelt', 'man', {})
('1945_Franklin_D._Roosevelt', 'peace', {})
('1945_Franklin_D._Roosevelt', 'test', {})
('1945_Franklin_D._Roosevelt', 'today', {})
```

Projection



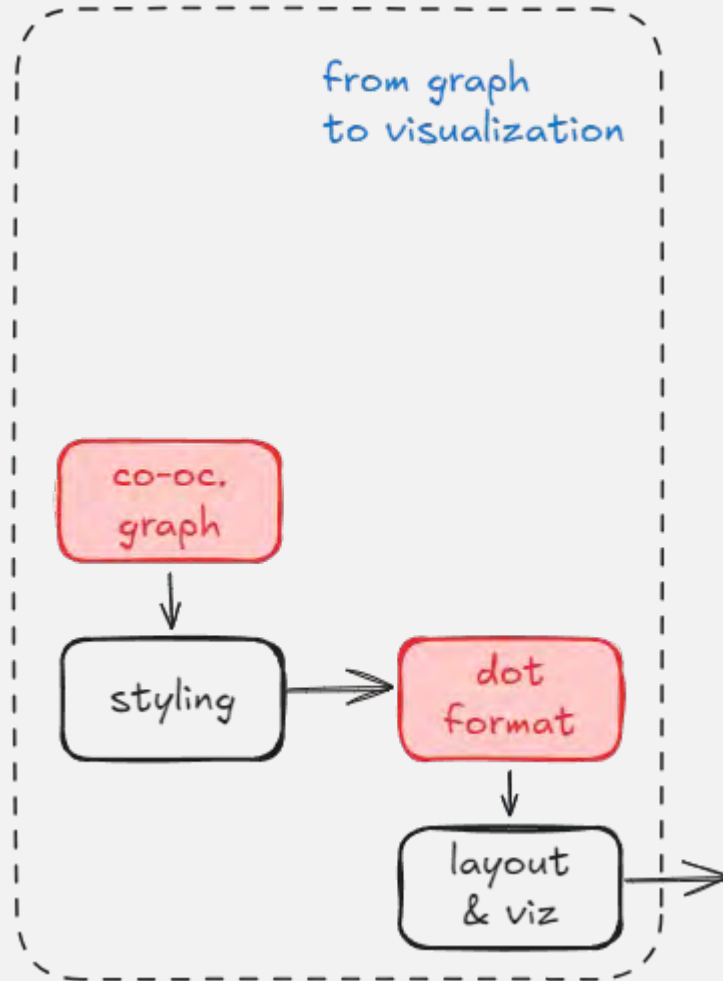
Projecting the Bipartite Graph

```
G = bipartite.weighted_projected_graph(B, [n for n,att in B.nodes(data=True) if att['type'] == 'term'])
```

or

```
G = bipartite.weighted_projected_graph(B, [n for n,att in B.nodes(data=True) if att['type'] == 'address'])
```

Finally ...



A change of view

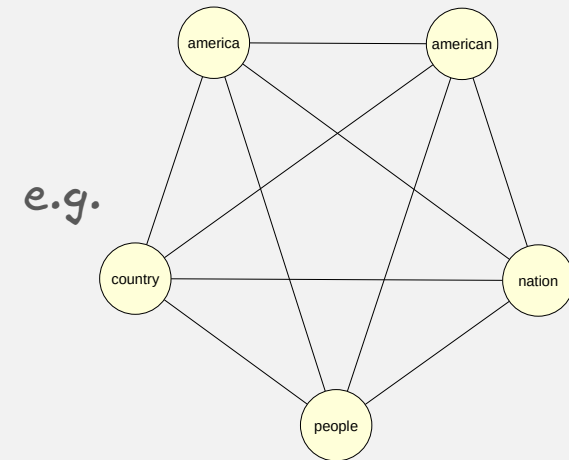
So far in this talk:

- graphs (nodes and edges) are abstract mathematical objects (combinatorial structures)

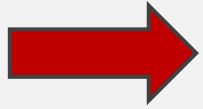
e.g. $G = (V, E)$

Now:

- nodes and edges are visual geometrical objects, having
 - positions in 2D space, and
 - attributes like size, color, borders, inscriptions, ...



And the Question is ...



Can we design these "visuals" such that they support our research?

Node Placement

aka. graph layout

Layout algorithms

sfdp	Scalable force-directed (see fdp below)
neato	places a virtual ideal spring between every single pair of nodes in the graph —not just the ones that share an edge. Node placement by global stress minimization
dot	graph drawing in which the vertices of a directed graph are drawn in horizontal rows with the edges generally directed downwards.
fdp	nodes interact via repulsive and attractive forces. Goal: reducing system forces to find a balanced layout
twopi	Radial / circular layout

Node Styling

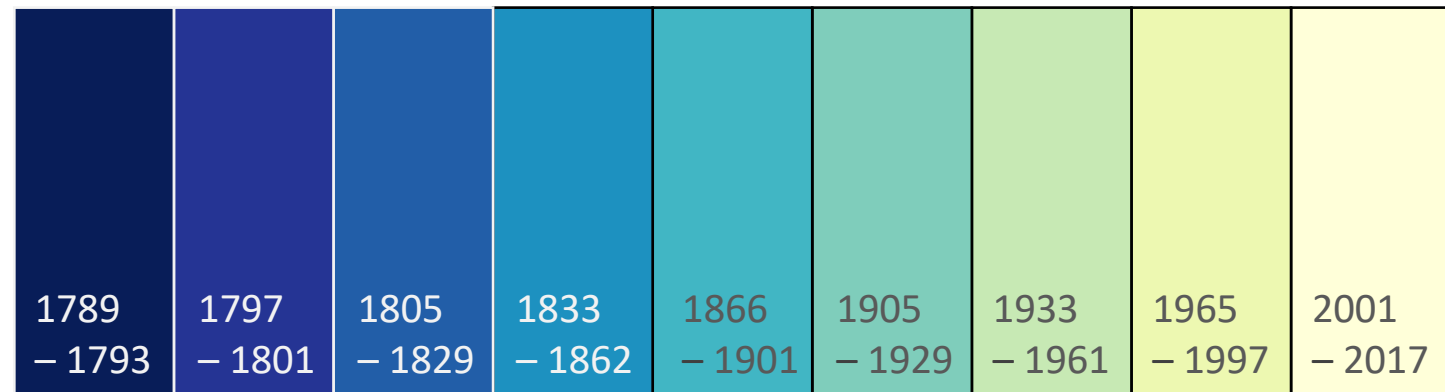
Node Coloring

By party:

-  Republican
-  Democrat
-  Both parties

By year of first use:

Epochs mapped to 9-class blue-to-yellow palette entries



Node Styling (2)

Node Sizing

Metric	Meaning
doc_freq	# addresses containing the term*
degree	# connected terms/addresses
tf-idf	Max TF-IDF score for term across addresses*

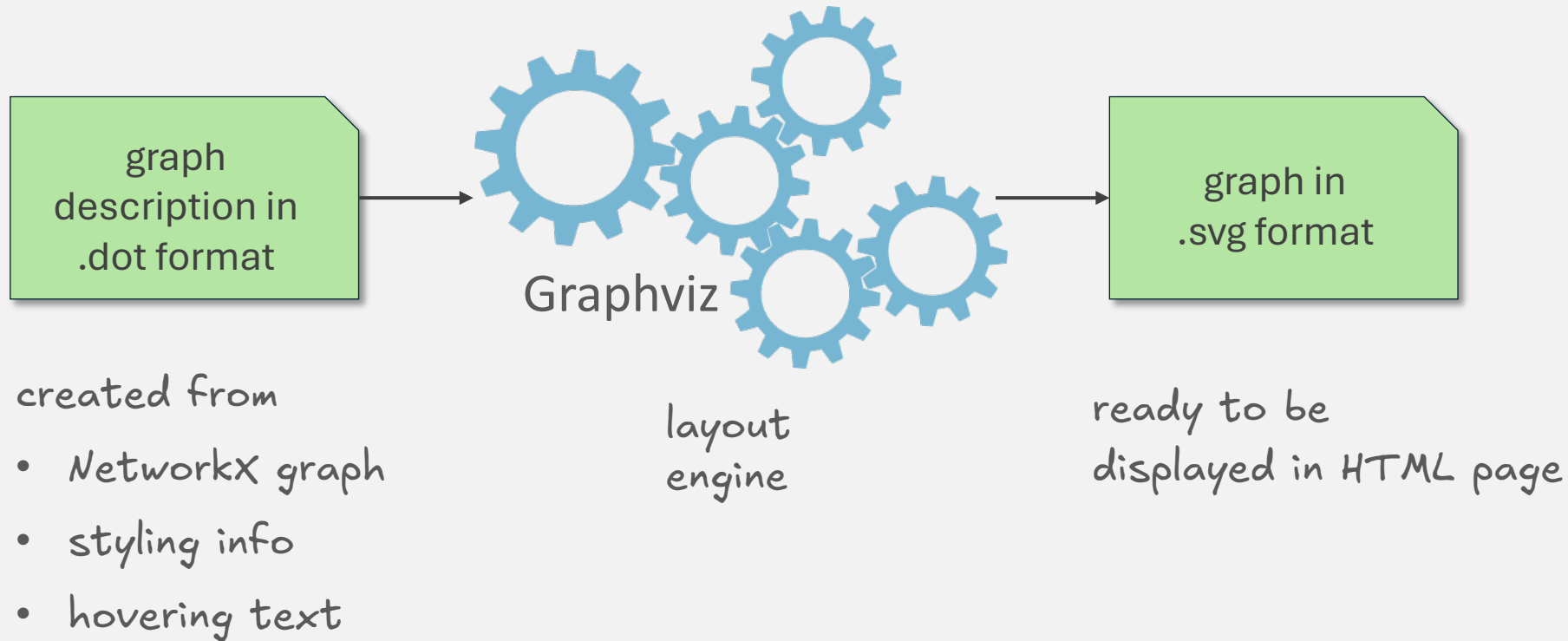
* not available in address graph

Node Styling ⁽³⁾

Node Size Growth

	Meaning
proportional	$a + c \cdot metric$
radix	$a + c \cdot \sqrt{metric}$

Putting it together



dot Language

```
graph {  
    // graph attributes  
    layout=neato;           // Layout algorithm  
    overlap=false;         // Prevent nodes from overlapping  
    splines=true;          // Route edges smoothly around nodes  
  
    // default attributes for nodes  
    node [margin=0 fontname="Arial" fontcolor="black" fontsize=64 shape=circle style=filled fixedsize=true];  
  
    // nodes with attributes  
    a [label="Foo"]  
    b [shape=box]  
  
    // edges with attributes  
    a -- b -- c [color=blue]  
    b -- d [style=dotted]  
}
```

Creating dot

```
def graphToDot(graph=None, node_coloring="by year of first use", node_sizing="doc_freq", node_size_growth="proportional"):  
    #dot file header  
    dot = 'graph {\n    overlap="prism1000"\n    rankdir="LR"\n    outputorder="edgesfirst"\n    splines="false"\n    '  
    dot += '    bgcolor="silver"\n    fontsize="60"\n    fontname="Arial"\n    labelloc="t"\n    labeljust="l"\n    '  
    dot += '    node [margin=0 fontname="Arial" fontcolor="black" fontsize=64 shape=circle style=filled fixedsize=true];\n'  
    # edges  
    for u,v,att in graph.edges(data=True):  
        dot += f'    {u} -- {v} [id="{u}--{v}" '  
        dot += f'    penwidth= {computeEdgePenwidth(att)} '  
        dot += f'    color   ="{computeEdgeColor(att)}"]\n'  
    #nodes  
    for u,att in graph.nodes(data=True):  
        dot += f'    {u} [id="{u}" '  
        dot += f'    fillcolor="{computeNodeFillColor(att, node_coloring)}" '  
        dot += f'    fontcolor="{computeNodeFontcolor()}" '  
        dot += f'    width      = {computeNodeWidth(att, node_sizing, node_size_growth)} '  
        dot += f'    tooltip  ="{computeNodeTooltip(att)}"]\n'  
    # close dot string  
    dot += '}'  
    return dot
```

Node Styling (3)

Node Size Growth

```
def width(metric, prop_factor, growth, min_width):
    if growth == "proportional":
        return max(min_width, prop_factor*(metric))
    elif growth == "radix":
        return max(min_width, 4*prop_factor*math.sqrt(metric))
    else:
        st.error("Invalid node size growth option selected.")

def computeNodeWidth(att, node_sizing, node_size_growth):
    if att['type'] == 'address':
        return width(int(att.get('degree')), 8.0, node_size_growth, 64)
    elif att['type'] == 'term':
        if node_sizing == "doc_freq":
            return width(int(att.get('doc_freq')), 8.0, node_size_growth, 48)
        elif node_sizing == "degree":
            return width(int(att.get('degree')), 8.0, node_size_growth, 64)
        elif node_sizing == "tf-idf":
            return width(float(att.get('tf_idf')), 16.0, node_size_growth, 48)
        else:
            st.error("Invalid node sizing option selected.")
            return
    else:
        st.error("Invalid node type.")
```

Dashboard Overview



What It Does

Analyzes U.S. presidential inauguration addresses (1789–2017) through NLP and graph theory

- Let's the user build **corpus**es by selecting subsets of addresses by epoch and political party
- Extracts keywords via TF-IDF, builds keyword **and** address co-occurrence graphs, and visualizes them as interactive graphs
- Displays some graph related **statistics** diagrams

Three-column Responsive Layout

Left (3)

None
Party
All

Graph construction and visualization
Keyword score

Data Selection

- Year range slider
- History period selectbox

Party selectbox

Graph Construction

- Radio buttons: keyword/address graph
- Keyword score slider

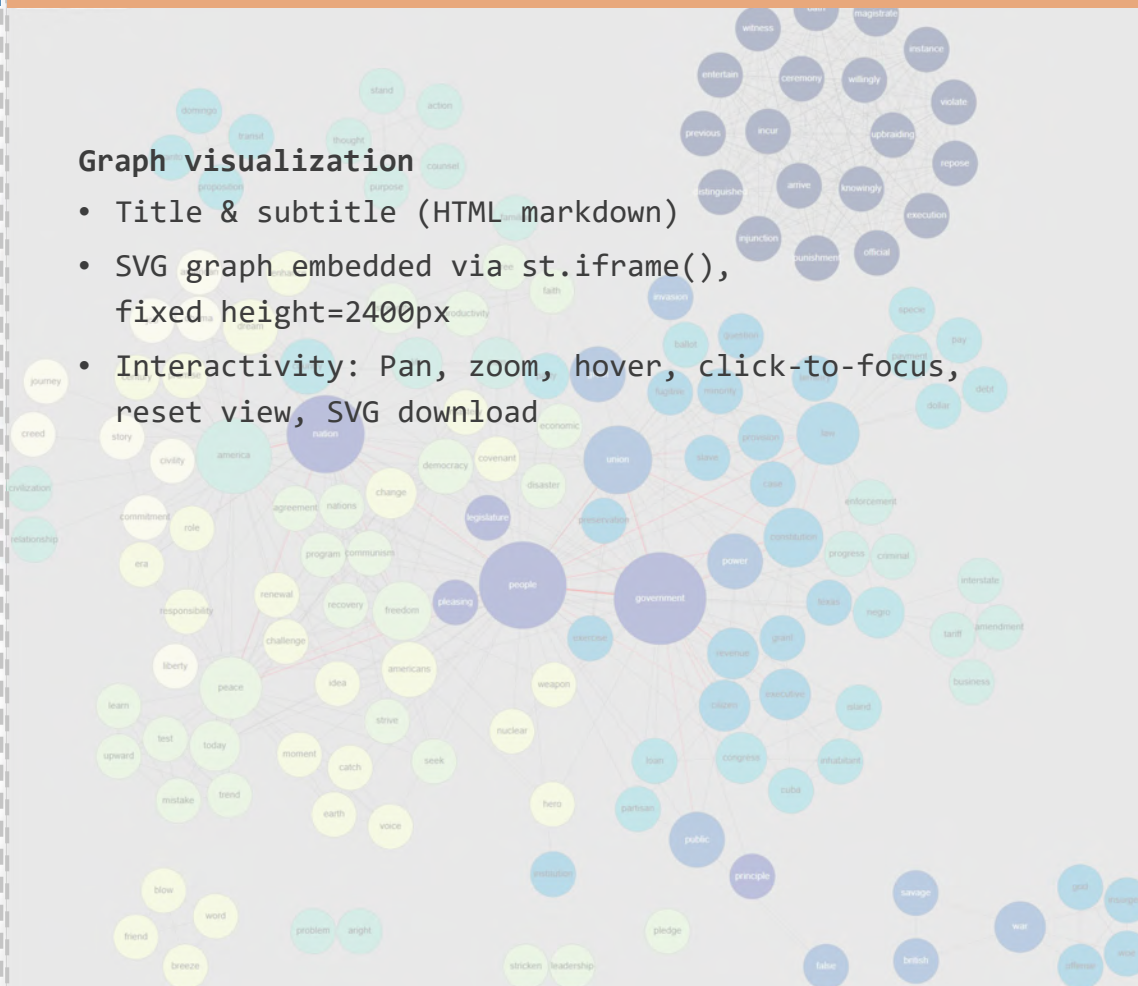
Viz Parameters

- Node coloring
- Node sizing / growth
- Layout engine

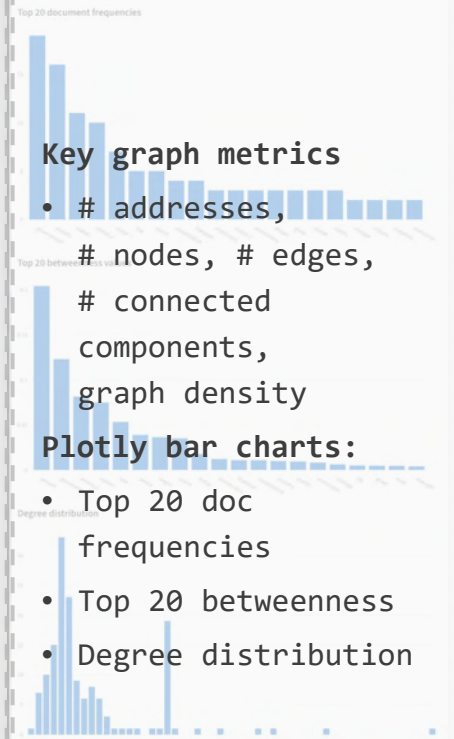
Center (10)

Graph visualization

- Title & subtitle (HTML markdown)
- SVG graph embedded via `st.iframe()`, fixed height=2400px
- Interactivity: Pan, zoom, hover, click-to-focus, reset view, SVG download



Right (3)



Interactive SVG Features



Pan & Zoom

- Mouse drag to pan the graph view
- Scroll wheel zooms in/out centered on cursor position



Click-to-Focus

Click a node to highlight it and its direct neighbors; all other nodes fade to 15% opacity



SVG Download

One-click export: serializes current SVG from DOM, adds XML header, triggers browser download



Reset View

Restores original viewBox and clears any active focus mode; also re-centers on window resize

Technology Stack



Streamlit

Web UI framework:
layout, widgets, forms,
session state, iframe embedding



MongoDB Atlas

Cloud database storing inauguration
address documents together with tags
(year, party, text)



spaCy + sklearn

NLP: spaCy lemmatization,
TF-IDF vectorization via scikit-learn



NetworkX

Graph construction: bipartite graphs,
weighted projection, graph metrics



PyGraphviz

Graph layout engines (sfdp, neato,
dot, fdp, twopi), node and edge
styling, and SVG rendering



Plotly

Bar charts for document frequency,
betweenness, degree distribution

Thank you!

The Address Repository

U.S. presidential inauguration addresses are historic speeches delivered by incoming presidents to unify the nation, outline their administration's vision, and address current crises.

Since 1789, these 63 addresses have evolved from indoor speeches to major outdoor events at the Capitol.

George Washington (1793): Shortest address at only 135 words.

William Henry Harrison (1841): Longest address at 8,455 words, lasting nearly two hours.

Abraham Lincoln (1861/1865): Focused on avoiding war and later, reconciling the nation with "malice toward none; with charity for all".

Franklin D. Roosevelt (1933): Famously declared during the Great Depression that "the only thing we have to fear is fear itself".

John F. Kennedy (1961): Challenged citizens with "ask not what your country can do for you—ask what you can do for your country".

Ronald Reagan (1981): Declared that "government is not the solution to our problem; government is the problem".

AI-generated overview (Gemini)